Software Requirements Specification (SRS)

1. Introduction

Purpose and Scope

The purpose of this document is to define the software requirements for FlowRoute, an automated workflow and resource manager. FlowRoute aims to eliminate the "Single-Point-of-Failure" in video content production by replacing manual assignment processes with a data-driven routing algorithm. The scope encompasses a Client Intake Portal for raw asset submission, a Smart Routing Engine for workload-balancing (calculating the lowest active task count), and a real-time Pod Workload Dashboard for administrative oversight.

Definitions, Acronyms, Abbreviations

- **PO:** Product Owner (Manages client relationships and administrative overrides).
- **Pod:** A designated video editing team.
- **DFD:** Data Flow Diagram.
- **FR / NFR:** Functional Requirement / Non-Functional Requirement.

References

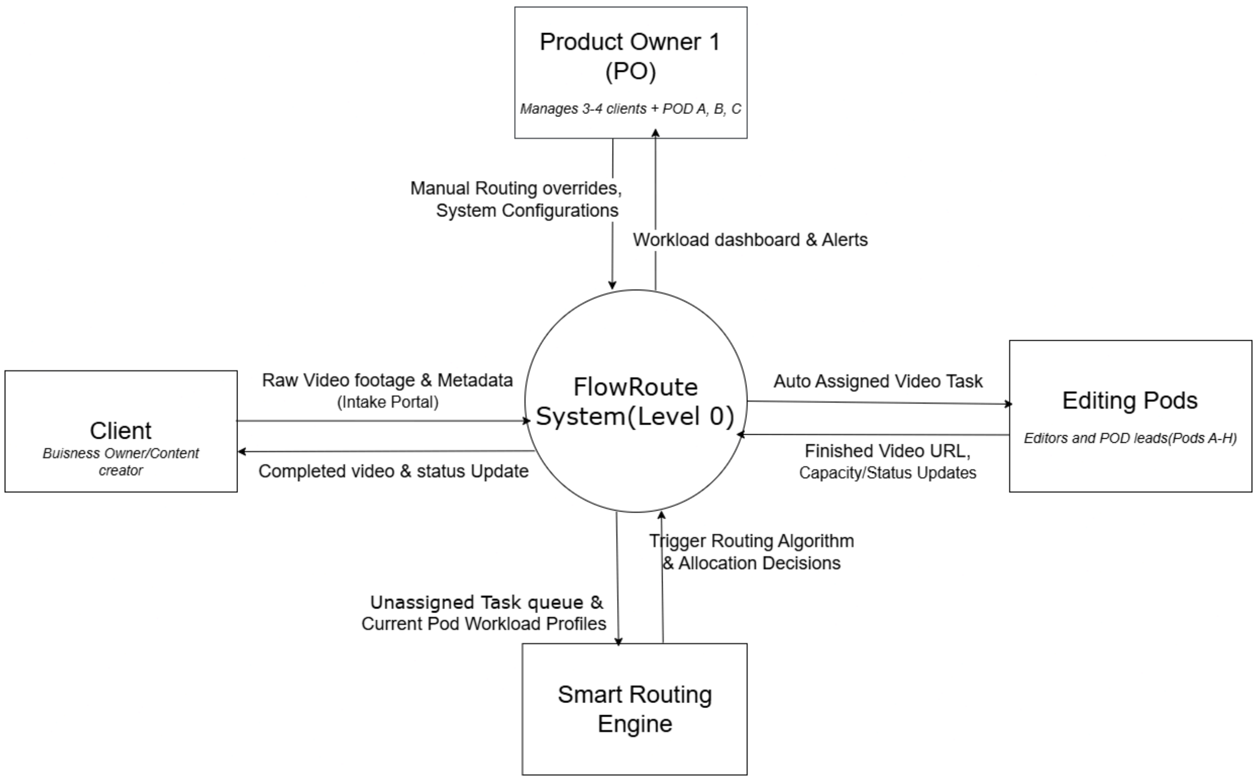
- Unwir Agency Fact-Finding Interview Documentation

Overview of Document

This document outlines the overall product perspective, analyzes the existing manual system, details the Unified Modeling Language (UML) structural models, and the specific functional and non-functional requirements governing the FlowRoute architecture.

2. Overall Description

Product Perspective
FlowRoute operates as a centralized web application facilitating communication between external clients, internal editing pods, and agency administrators.



User Classes and Characteristics

- **Clients (Content Creators/Brands):** Require a frictionless, intuitive upload portal without the complexity of enterprise developer tools.

- **Product Owners (e.g., Sankhaja from Unwir):** Need high-level dashboard visibility to monitor bottlenecks and the authority to manually override automated routing when exceptions occur.

- **Editing Pods:** Require clear task queues and fair workload distribution, without needing to manage client interactions.

Operating Environment

The system will operate as a cloud-hosted web application. The frontend will be accessible via standard web browsers. The backend will execute Node.js REST APIs and Python compute logic, interfacing with a MongoDB database.

Design and Implementation Constraints

The project must adhere to strict budget constraints using student/free-tier cloud infrastructure (e.g., AWS/Vercel). Consequently, video asset uploads are strictly capped at 50 Megabytes (50MB), and automated cloud-storage purges will occur every 7 days.

Assumptions and Dependencies

It is assumed that Editing Pods have stable internet connections to receive real-time WebSocket/Polling updates and that clients can output raw video files within the 50MB constraint.

3. System Analysis

Fact-Gathering Techniques Used

To establish the system requirements, identify core bottlenecks, and define the automated routing architecture, the team conducted two distinct fact-finding sessions:

1. Structured Stakeholder Interview

- **Target Participant:** Sankhaja (Founder & Product Owner, Unwir Agency)
- **Date:** 23 May 2026
- **Methodology:** A 45-minute semi-structured interview conducted via Google Meet, utilizing a 10-question guide focusing on intake, evaluation, time costs, and metrics.
- **Key Findings:**
    - The current intake process relies on fragmented tools (Emails, Google Sheets, Trello), resulting in outdated capacity tracking.
    - The Product Owner wastes 10–15 hours weekly on manual "traffic control," creating a single point of failure. Videos submitted when the PO is offline experience a 24-hour+ delay.
    - Workload distribution is currently based on "gut feeling," causing severe imbalances where some pods work 12-hour days while others remain idle.

2. Joint Application Design (JAD) Workshop

- **Participants:** Internal Development Team (Pabodha, Sanira, Hansi, Chathura)
- **Date:** 22 May 2026
- **Methodology:** Internal collaborative session (via Discord/Teams) facilitated to translate stakeholder pain points into logical system decisions.
- **Key Decisions & Technical Findings:**
    - **Tie-Breaker Logic:** If pods have identical task counts, tasks will route to the pod with the fastest rolling 7-day average completion time.
    - **Capacity Thresholds:** A pod is operating at maximum capacity when its active task count equals its total assigned editors multiplied by two.
    - **Fail-Safe Queue:** If all pods reach maximum capacity, overflow tasks will be held in a Master Pending Queue.
    - **UI/UX Strategy:** The PO dashboard requires a real-time bar chart comparing all 8 pods to instantly visualize standard deviation imbalances.

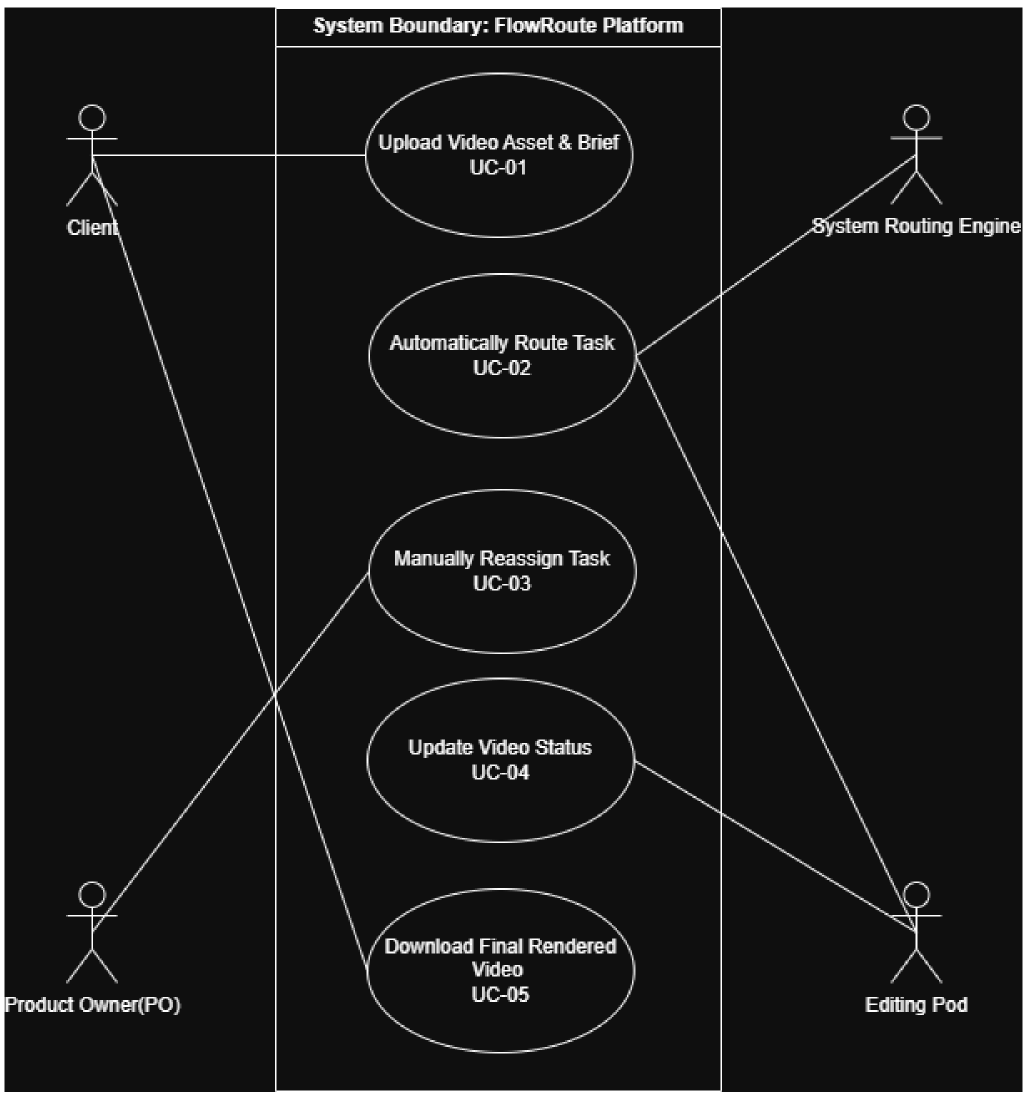
Existing System Analysis (Pain points)

The existing agency workflow is highly fragmented and entirely manual. Current pain points include:

- **Pain Point 1:** 100% manual assignment creates a severe bottleneck; tasks stall completely when the PO is offline or in meetings.
- **Pain Point 2:** A lack of real-time capacity tracking leads to severe workload imbalances and burnout for specific editing pods.
- **Pain Point 3:** The disconnected pipeline between Email, Google Sheets, and Trello is highly prone to human error, as editors frequently forget to manually update tracker statuses.

UML Models

- **Use Case Diagram:**



- **Use Case Descriptions:**

UC-01: Client Video Intake

Field	Content
Use Case ID	UC-01
Use Case Name	Client Video Intake
Actor(s)	Client, FlowRoute System
Preconditions	Client has a registered account and is logged in to the portal; client has a video file and brief document ready to upload.
Postconditions - Success	Video files and briefs are stored in the system; the FlowRoute processing pipeline is triggered; client receives a submission confirmation.
Postconditions - Failure	Files are not stored; pipeline is not triggered; an error message is displayed to the client.
Main Flow	1. Client navigates to the submission portal and clicks Upload. 2. System displays the file upload form. 3. Client selects a video file from their device. 4. System validates the file format and checks it is within the allowed size limit. 5. Client attaches the brief document. 6. Client clicks Submit. 7. System stores the video file and brief. 8. System triggers the FlowRoute processing pipeline. 9. System displays a submission confirmation to the client.
Alternative Flow 1	At step 4: If the file fails validation (wrong format or exceeds size limit), the system displays an error message describing the reason. The client corrects the issue (e.g., compresses or reformats the file) and re-selects the file, returning to step 3.
Alternative Flow 2	At step 6: If the client clicks Cancel before submitting, the system discards any uploaded files and returns the client to the portal home page. The pipeline is not triggered.
Business Rules	BR-01: Accepted video formats are MP4, MOV, and AVI. BR-02: Maximum video file size is 2 GB. BR-03: A brief document must be attached before submission is allowed.

UC-02: Automated Task Routing

Field	Content
Use Case ID	UC-02
Use Case Name	Automated Task Routing
Actor(s)	System (Routing Engine), Editing Pods
Preconditions	A raw video asset and a matching client brief have been successfully processed and verified in the Intake Portal (UC-01).
Postconditions - Success	The video task is assigned to the Editing Pod with the lowest relative active workload, and team notifications are successfully dispatched.
Postconditions - Failure	The video task is automatically flagged and placed into the master queue awaiting manual supervisor intervention.

Main Flow	1. The System detects a newly finalized video task entry in the primary intake queue. 2. The System queries the database to retrieve the active task count and member size for all 8 Editing Pods. 3. The System calculates the current capacity ratio for each individual pod. 4. The System identifies the specific Editing Pod possessing the lowest active workload ratio. 5. The System assigns the video task ID to the identified Editing Pod. 6. The System updates the Pod Workload Dashboard in real-time. 7. The System dispatches an internal notification alert via Webhook to the assigned Editing Pod's channel.
Alternative Flow 1	At step 4: If the System determines that all 8 Editing Pods have reached or exceeded their designated maximum active task limits, the system sets the video task status to "Queued", triggers a high-priority system alert to the Product Owner indicating a critical capacity bottleneck, and the use case terminates with the task remaining in the master queue.
Alternative Flow 2	At step 4: If two or more qualifying Editing Pods share the exact same lowest active workload ratio, the System queries historical metrics to analyze the rolling 14-day completion time of the tied pods, selects and assigns the task to the tied pod with the fastest average turnaround time, and the flow automatically resumes from step 5.

Business Rules	**BR-01:** The System shall not allocate an automated task to any Editing Pod whose active task count meets or exceeds its maximum threshold as defined by the Product Owner. **BR-02:** Workload metrics must be computed dynamically using the total active task load divided by the number of active editors assigned to that specific pod, preventing skewed distributions between unequal team sizes. **BR-03:** The historical turnaround performance calculation utilized in Alternative Flow 2 must be based strictly on a rolling 14-day window of completed tasks.

UC-03: Manual Task Reassignment

Field	Content
Use Case ID	UC-03
Use Case Name	Manual Task Reassignment
Actor(s)	**Primary Actor:** Product Owner (PO) **Secondary Actor(s):** Editing Pods, Notification Service

Preconditions	1. The Product Owner is securely authenticated into the administrative dashboard. 2. The automated routing engine (UC-02) has flagged a capacity bottleneck or a manual override is requested.
Postconditions - Success	1. The video task is updated with a forced Editing Pod assignment. 2. The real-time Pod Workload Dashboard reflects the updated queue state immediately.
Main Flow	1. The Product Owner logs into the administrative dashboard interface. 2. The PO selects an active, blocked video task from the bottleneck queue view. 3. The PO selects an alternative target Editing Pod to handle the workload. 4. The PO clicks the "Force Reassign" button. 5. The system processes the administrative assignment override and updates the database task record. 6. The system pushes a notification to the selected Editing Pod and updates the live Workload Dashboard metrics.

Alternative Flow A	**Target Pod Manual Rejection (AC2)** 1. At step 5 of the main flow, the selected target Editing Pod instance rejects the forced assignment (e.g., due to sudden technical hardware failure, node disconnection, or an absolute hard resource cap). 2. The system catches the assignment failure exception. 3. The system halts the transaction, rolls back the assignment state, and displays an error message: "Manual Assignment Failed: Target Pod rejected allocation." 4. The task retains its original queue state.
Alternative Flow B	**Task State Conflict** 1. If an internal editor completes and marks the video task as 'Rendered' while the PO is actively viewing the reassignment page, a state conflict occurs. 2. Upon the PO clicking "Force Reassign", the system detects the state modification. 3. The system aborts the override transaction and displays an informational message: "Reassignment Aborted: Task already completed."

UC-04: Update Video Status

Field	Content
Use Case ID	UC-04
Use Case Name	Update Video Status
Actor(s)	Editing Pod, FlowRoute System
Preconditions	The Editing Pod has an active video task currently assigned to it with a status of "Assigned"; the pod is authenticated and connected to the FlowRoute system.
Postconditions - Success	The video task status is updated in the system; the system's real-time workload capacity data is refreshed; if status is "Completed", the pod's active task count is decremented by 1.
Postconditions - Failure	The video task status remains unchanged; the system logs the failed update attempt; the pod is notified of the error.

Main Flow	1. Editing Pod completes its current processing stage on the assigned video task. 2. Pod sends a status update request to the FlowRoute system, setting the task status to "In Progress". 3. System receives the request and validates the pod's identity and task assignment. 4. System records the updated status as "In Progress" and timestamps the change. 5. Pod finishes processing and sends a second status update, setting the task status to "Review". 6. System records the updated status as "Review" and timestamps the change. 7. A reviewer approves the completed video output. 8. Pod sends a final status update, setting the task status to "Completed". 9. System records the status as "Completed", decrements the pod's active task count by 1, and updates the real-time workload capacity. 10. System confirms the update to the pod.
Alternative Flow 1	At step 3: If the system cannot verify the pod's identity or the task is not assigned to that pod, the system rejects the status update request and returns an authentication error. The pod is notified and no status change is recorded.

Alternative Flow 2	At step 3 or step 5 (during any status update transaction): If the system is temporarily unavailable or the update request times out, the pod retries the request up to 3 times with a 5-second interval. If all retries fail, the pod logs the error locally and alerts the system administrator. The task status remains at its last known valid state.
Business Rules	**BR-10:** Valid task status transitions are: "Assigned" → "In Progress" → "Review" → "Completed" only. No other transitions are permitted. **BR-11:** A pod may only update the status of tasks that are explicitly assigned to it. **BR-12:** Every status change must be timestamped and logged for audit and routing accuracy. **BR-13:** Decrementing the pod's active task count must occur only upon status reaching "Completed".

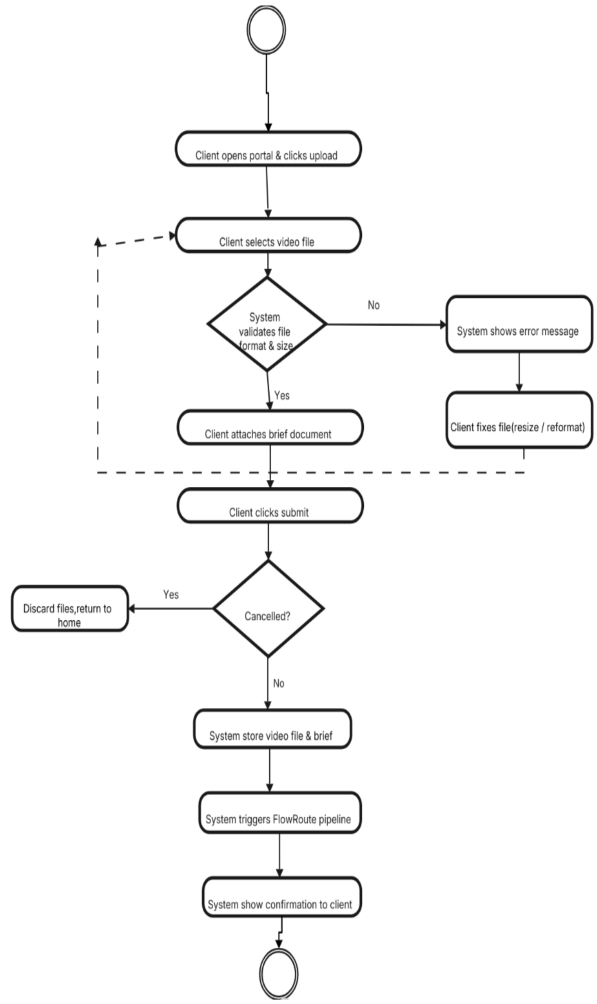
UC-05: Final Delivery Download

Use Case ID	UC-05
Use Case Name	Final Delivery Download
Actor(s)	Client, FlowRoute System
Preconditions	The video task status has been set to "Completed" (UC-04); the final rendered video file has been stored in the system and is ready for download; the client has a registered account and is logged in to the portal.
Postconditions – Success	The client successfully downloads the final rendered video file; the system marks the video asset's lifecycle as closed; a download event is logged with client ID, video ID, and timestamp.

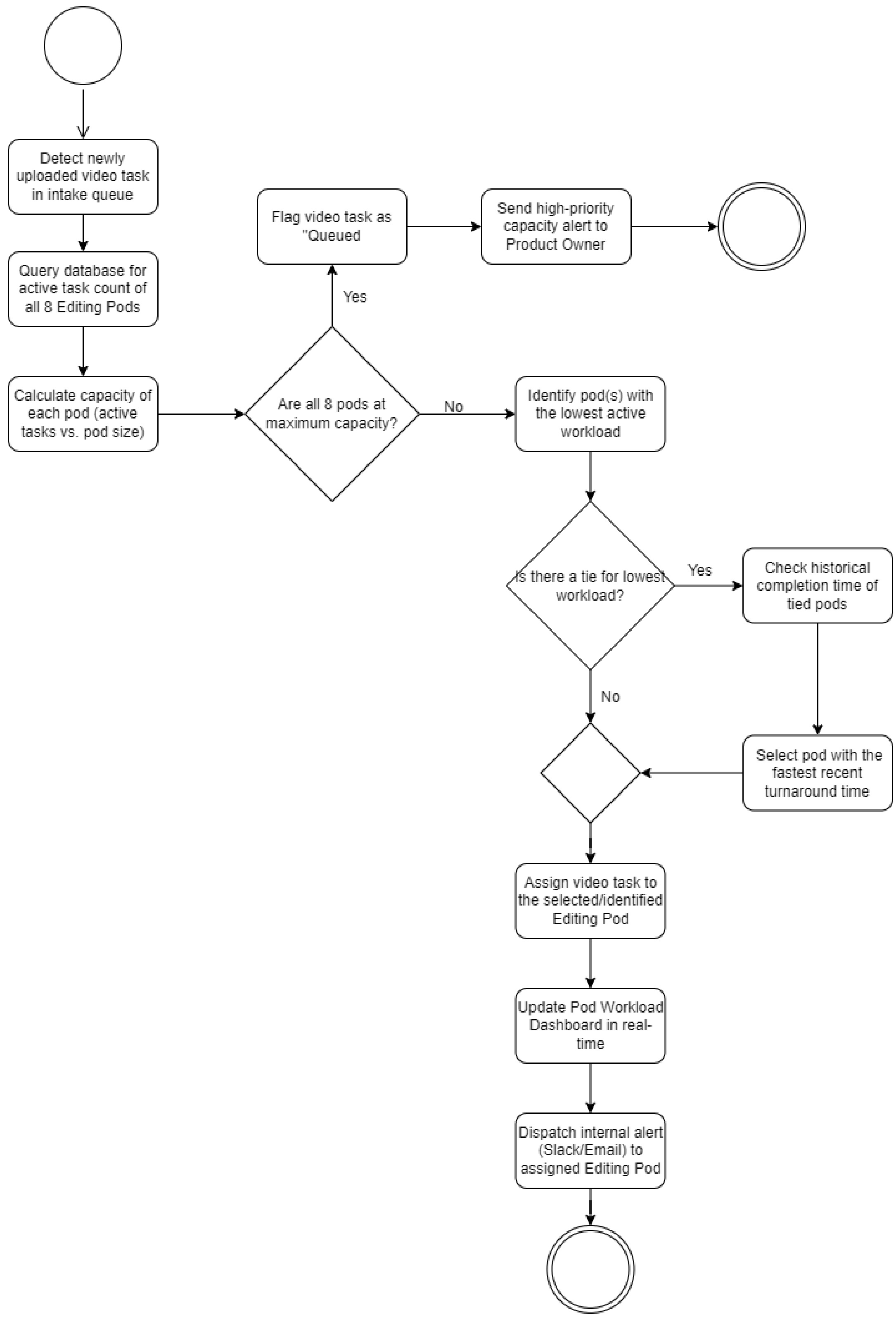
Postconditions – Failure	The video file is not downloaded; the system logs the failed attempt; the client is notified with the reason for failure.
Main Flow	1. System notifies the client that their final rendered video is ready for download. 2. Client logs in to the FlowRoute portal and navigates to their delivery dashboard. 3. Client clicks the Download button for the completed video asset. 4. System authenticates the client's identity and verifies they are the owner of the requested video asset. 5. System generates a secure, time-limited download link for the video file. 6. Client clicks the download link and the file transfer begins. 7. System logs the download event with client ID, video ID, and timestamp. 8. System marks the video asset lifecycle as closed upon successful download completion.
Alternative Flow 1	At step 4: If the client's authentication fails or they are not the verified owner of the asset, the system denies access and displays an authorisation error. The client is prompted to log in again or contact support. No download link is generated.
Alternative Flow 2	At step 6: If the download link has expired (link lifetime exceeded) before the client clicks it, the system displays an expiry message. The client may request a new download link, which returns the flow to step 5.
Alternative Flow 3	At step 6: If the file transfer fails midway (e.g., network interruption), the system retains the download link for the remainder of its valid period so the client can restart the download without repeating authentication.
Business Rules	BR-14: Download links are time-limited and expire after 24 hours of generation. BR-15: Only the verified owner client account may download a given video asset. BR-16: Each download event must be logged with client ID, video ID, and timestamp for audit purposes. BR-17: A video asset lifecycle is marked closed only after a successful, complete file transfer.

Activity Diagrams

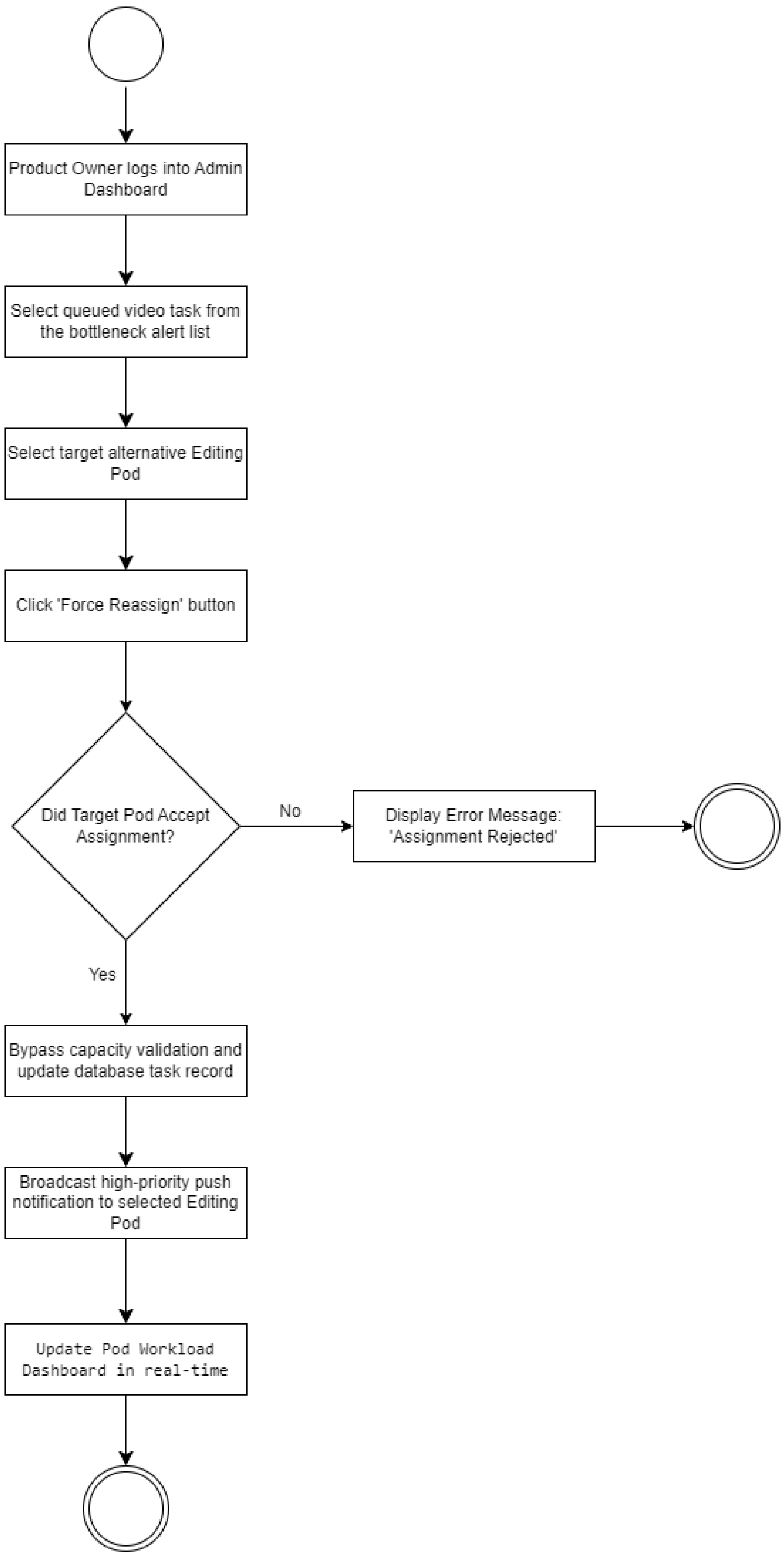
UC-01: Client Video Intake



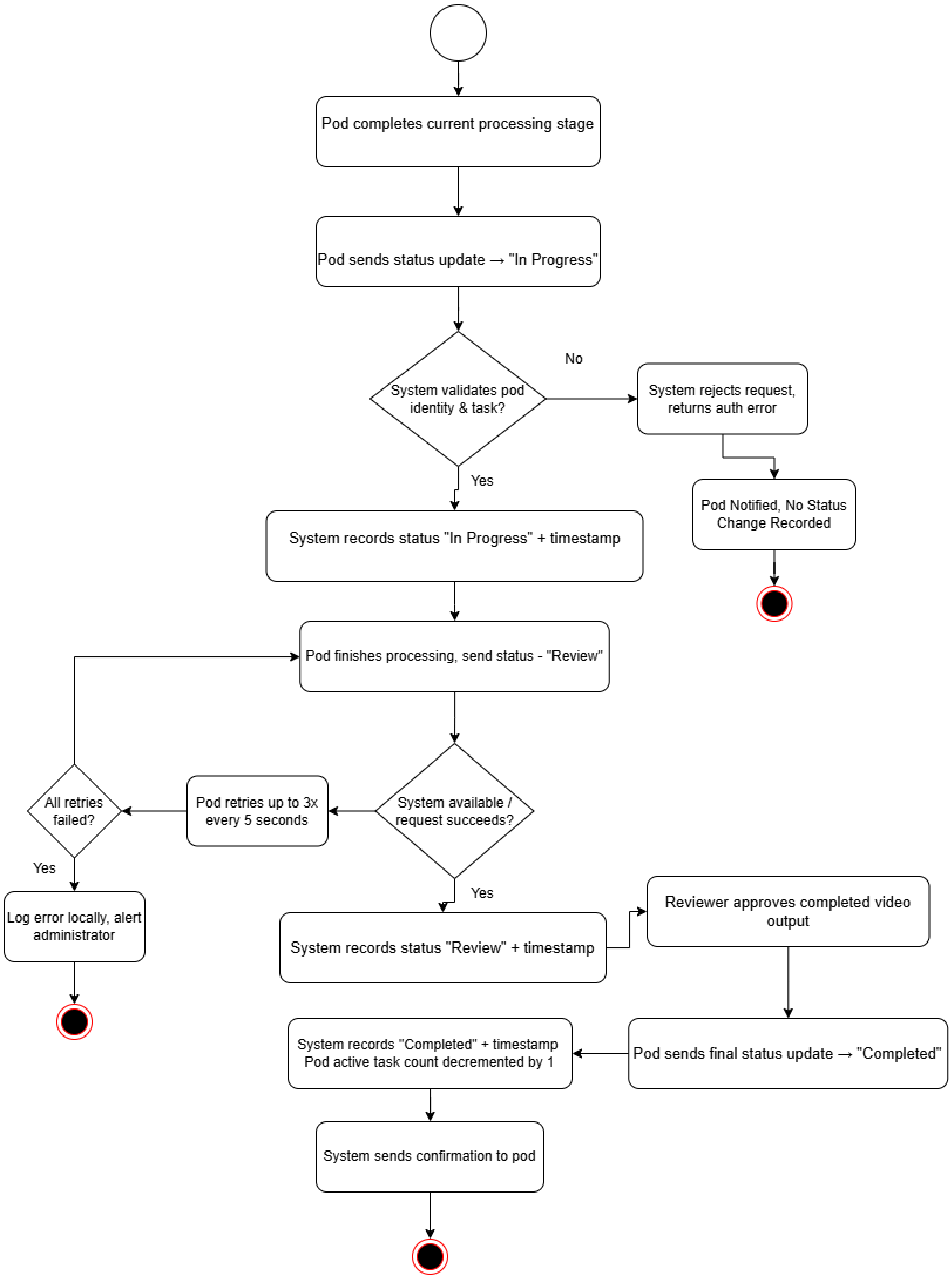
UC-02: Automated Task Routing



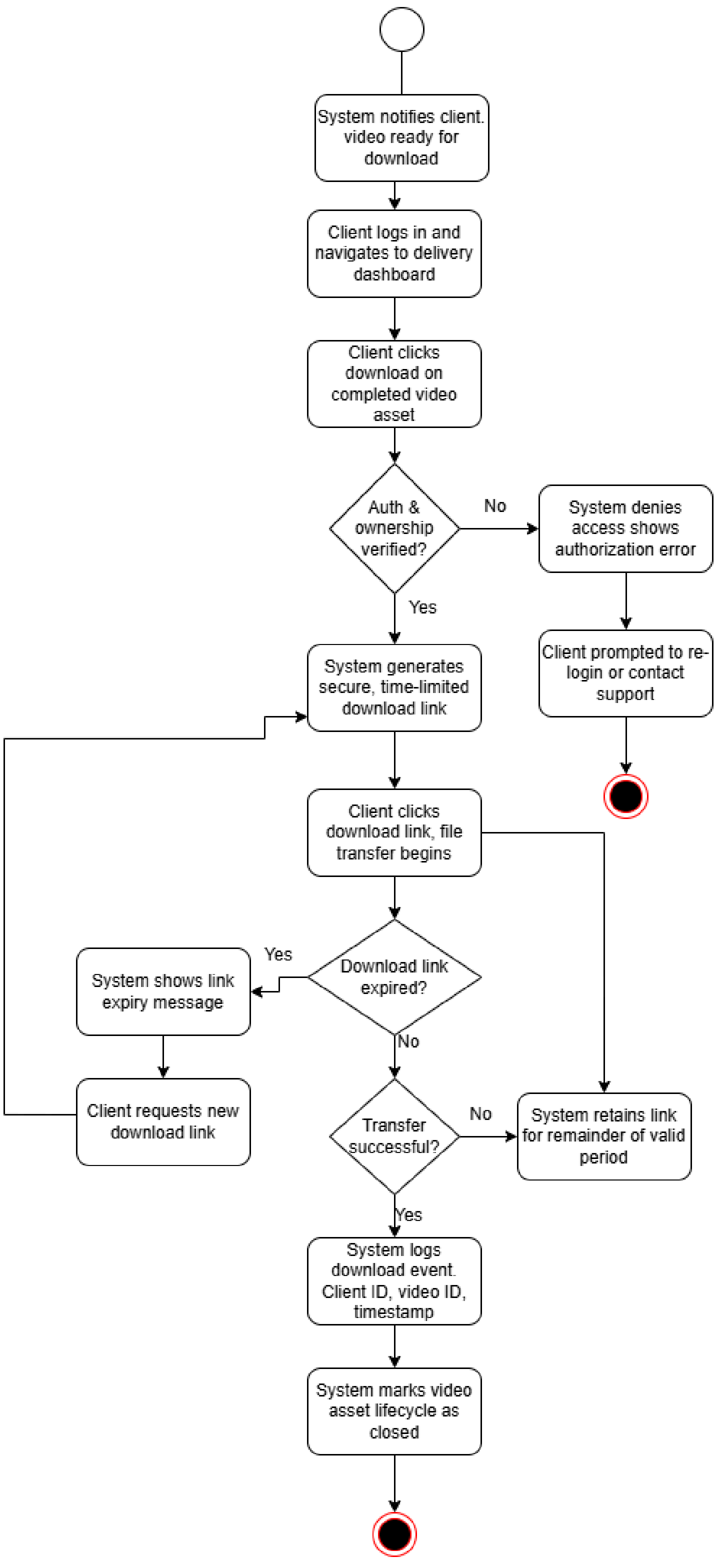
UC-03: Manual Task Reassignment



UC-04: Update Video Status

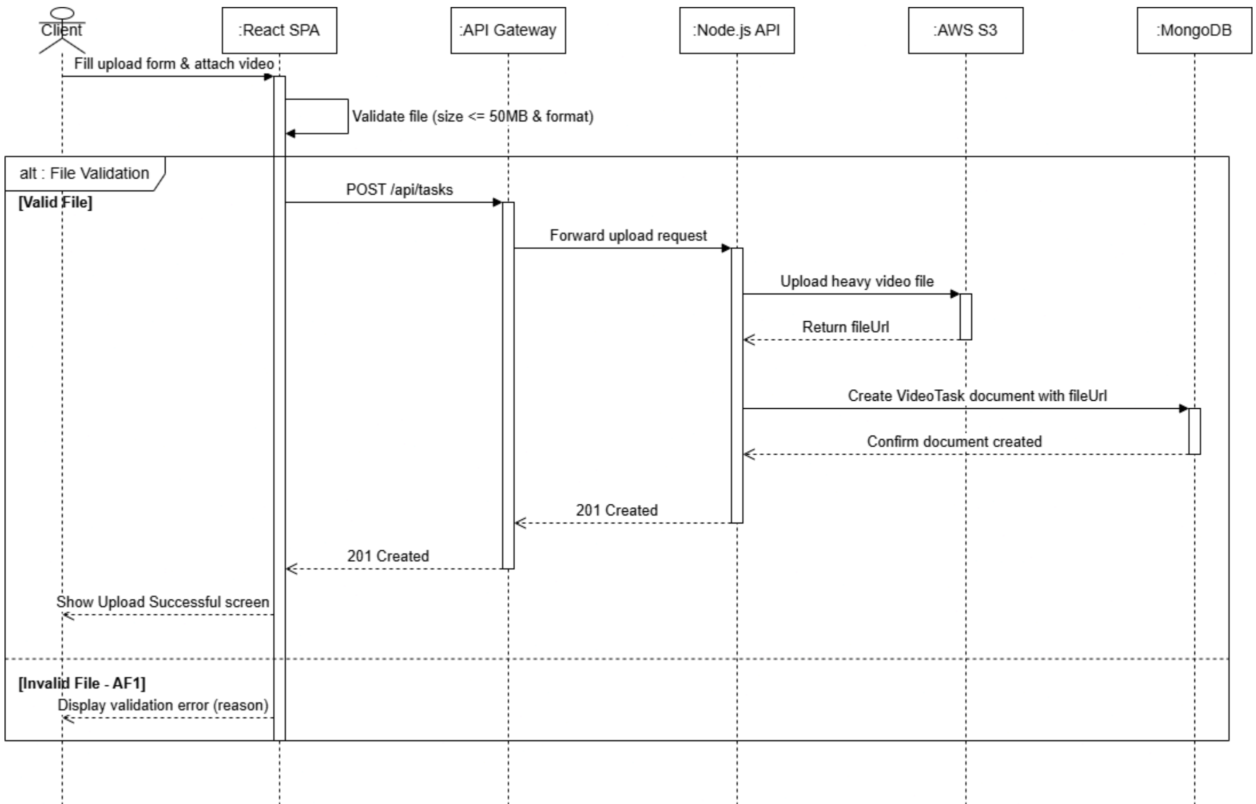


UC-05: Final Delivery Download

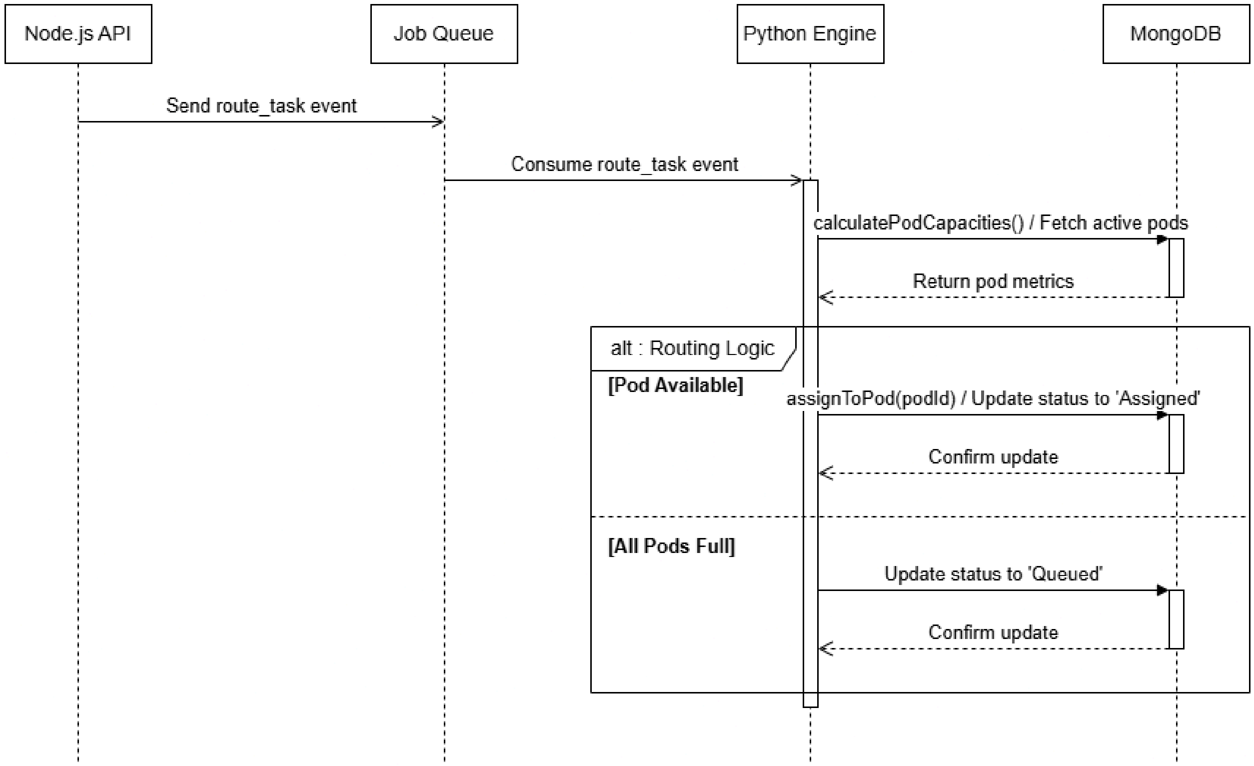


Sequence diagram

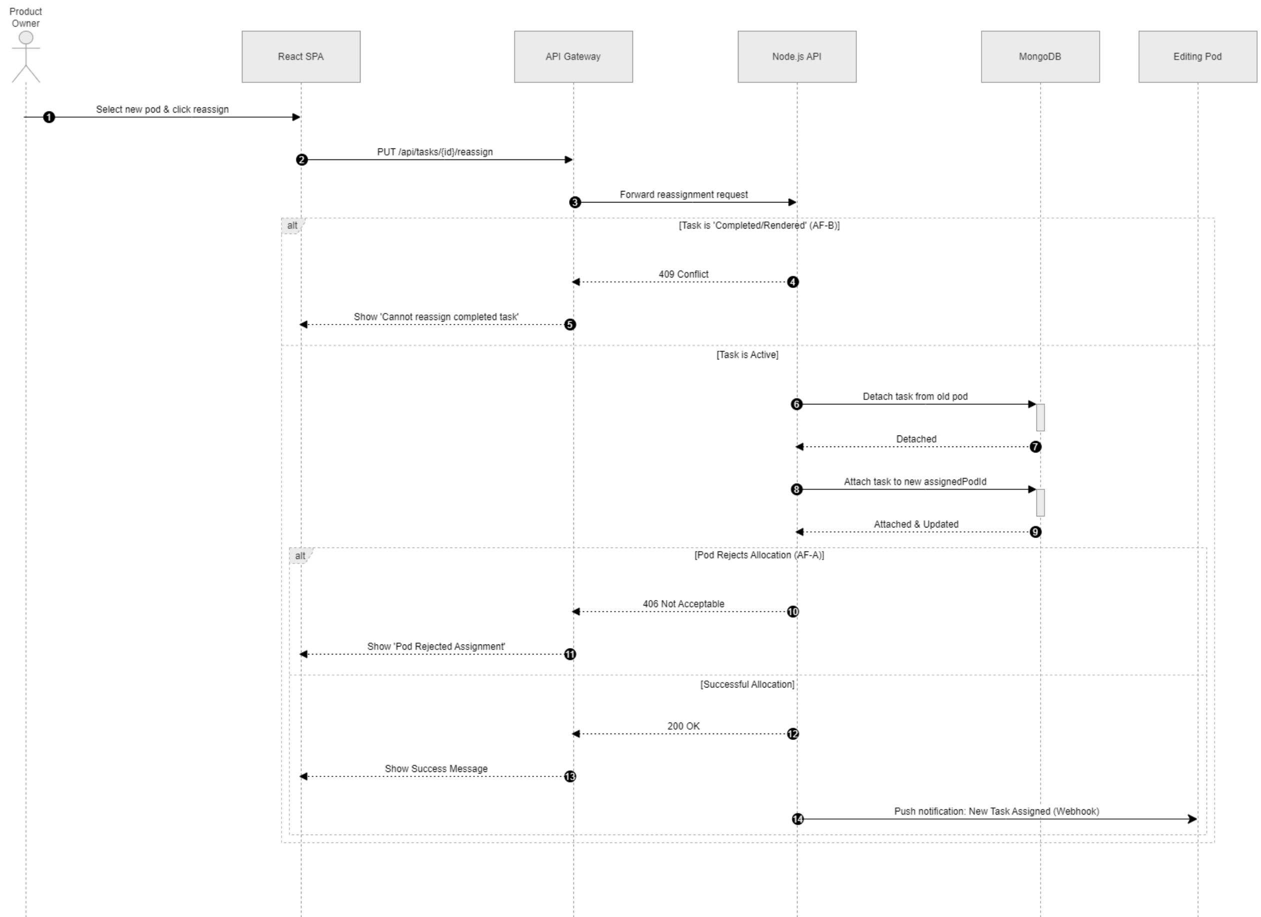
UC 01



UC 02

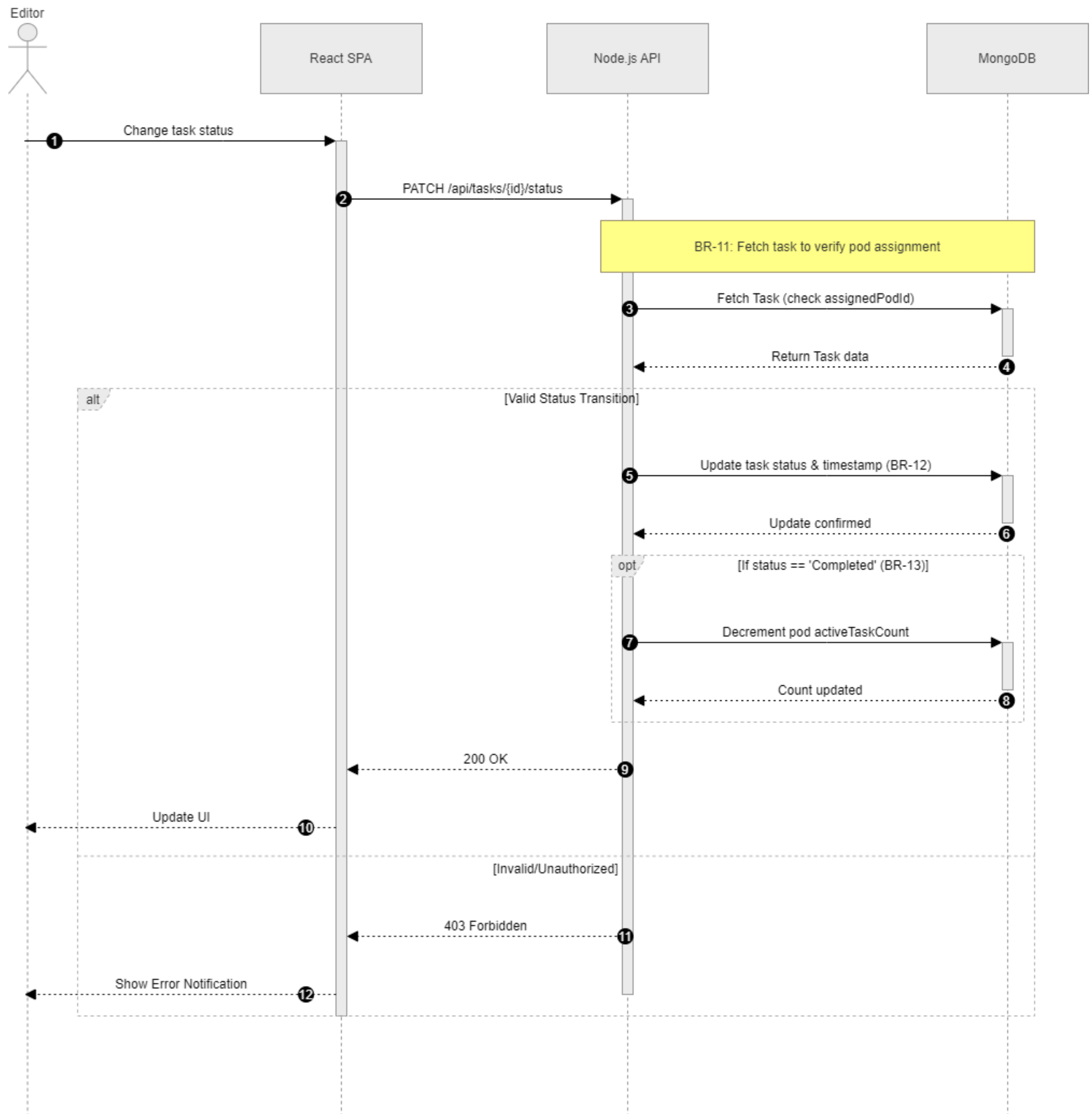

Node.js API
Job Queue
Python Engine
MongoDB
Send route_task event
Consume route_task event
calculatePodCapacities() / Fetch active pods
Return pod metrics
alt : Routing Logic
[Pod Available]
assignToPod(podId) / Update status to 'Assigned'
Confirm update
[All Pods Full]
Update status to 'Queued'
Confirm update

UC 03

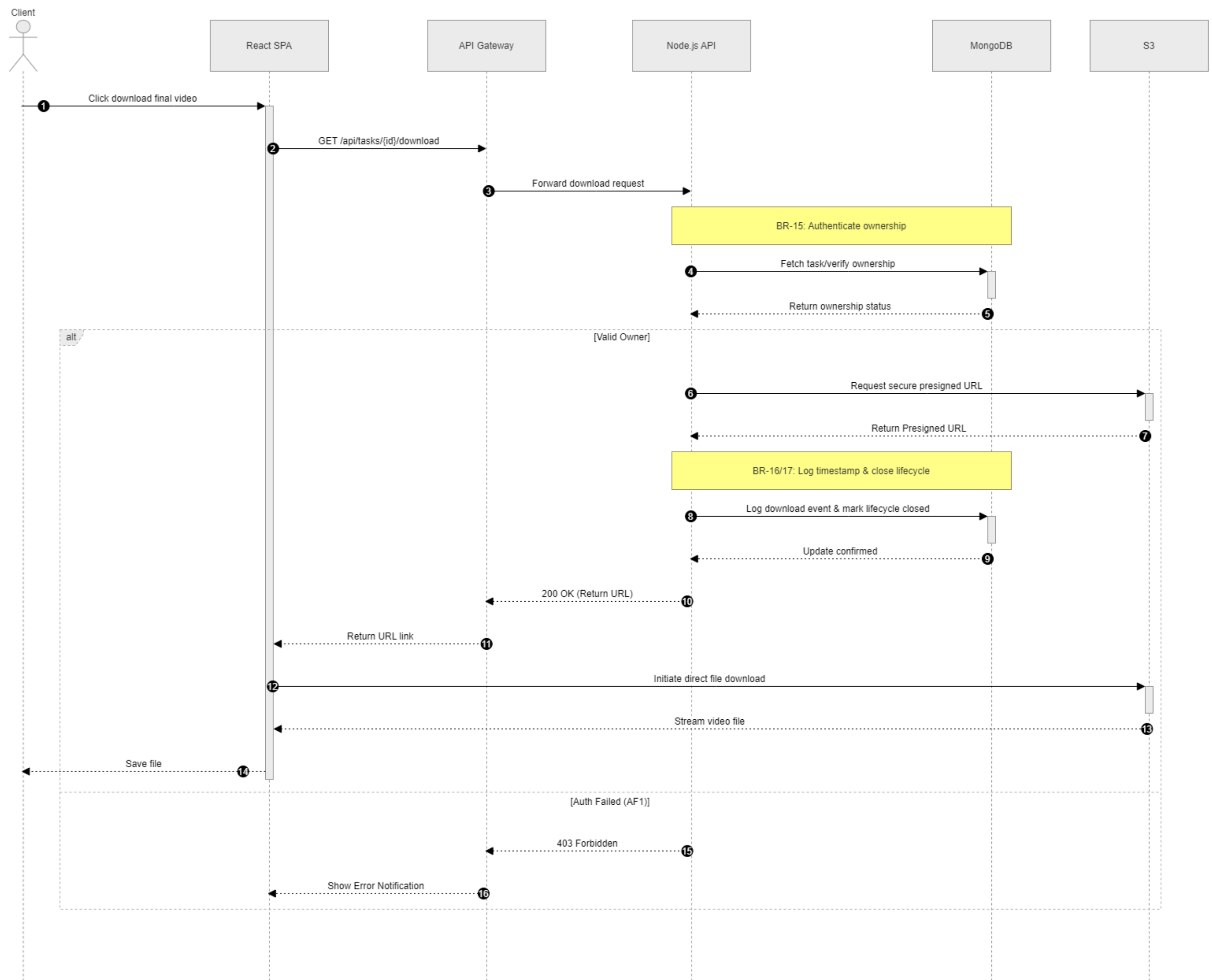


Product Owner
React SPA
API Gateway
Node.js API
MongoDB
Editing Pod
Select new pod & click reassign
PUT /api/tasks/{id}/reassign
Forward reassignment request
alt
[Task is 'Completed/Rendered' (AF-B)]
409 Conflict
Show 'Cannot reassign completed task'
[Task is Active]
Detach task from old pod
Detached
Attach task to new assignedPodId
Attached & Updated
alt
[Pod Rejects Allocation (AF-A)]
406 Not Acceptable
Show 'Pod Rejected Assignment'
[Successful Allocation]
200 OK
Show Success Message
Push notification: New Task Assigned (Webhook)

UC 04


Editor
React SPA
Node.js API
MongoDB
Change task status
PATCH /api/tasks/{id}/status
BR-11: Fetch task to verify pod assignment
Fetch Task (check assignedPodId)
Return Task data
alt
[Valid Status Transition]
Update task status & timestamp (BR-12)
Update confirmed
opt
[If status == 'Completed' (BR-13)]
Decrement pod activeTaskCount
Count updated
200 OK
Update UI
[Invalid/Unauthorized]
403 Forbidden
Show Error Notification

UC 05


Client
React SPA
API Gateway
Node.js API
MongoDB
S3
Click download final video
GET /api/tasks/{id}/download
Forward download request
BR-15: Authenticate ownership
Fetch task/verify ownership
Return ownership status
alt
[Valid Owner]
Request secure presigned URL
Return Presigned URL
BR-16/17: Log timestamp & close lifecycle
Log download event & mark lifecycle closed
Update confirmed
200 OK (Return URL)
Return URL link
Initiate direct file download
Stream video file
Save file
[Auth Failed (AF1)]
403 Forbidden
Show Error Notification

4. Functional Requirements

ID	Title	Description	Priority	Source	Verification
FR-01	Video File Upload	Raw video files shall be securely uploaded via the Client Intake Portal.	P1 - Blocker	UC-01	TC-FR-01
FR-02	Upload Constraints	Uploaded files validated against a max file size of 50 Megabytes (50MB).	P1 - Blocker	UC-01 / AC3	TC-FR-02
FR-03	Intake Confirmation	A confirmation displayed to the client upon successful upload/validation.	P3 - Major	UC-01	TC-FR-03
FR-04	Routing Trigger	Automated algorithm triggered immediately upon successful upload.	P1 - Blocker	UC-02 / AC3	TC-FR-04
FR-05	Capacity Retrieval	Active queue depth and real-time capacity of all pods retrieved prior to assignment.	P1 - Blocker	UC-02	TC-FR-05

FR-06	Automated Assignment	Task automatically assigned to the editing pod holding the lowest task count.	P1 - Blocker	UC-02	TC-FR-06
FR-07	Assignment Notification	Notifications dispatched to pod lead and PO once routed successfully.	P2 - Critical	UC-02 / AC3	TC-FR-07
FR-08	Manual Override	Routing manually overridden by PO during bottlenecks or deadlocks.	P1 - Blocker	UC-03	TC-FR-08
FR-09	Task Reassignment	Tasks manually re-assignable by PO to alternative pods for emergencies.	P2 - Critical	UC-03	TC-FR-09
FR-10	Routing Audit Logging	All assignment events logged to maintain verifiable workload history.	P3 - Major	UC-02	TC-FR-10

5. Non-Functional Requirements

5.1 Performance Requirements

- **NFR-PER-01 (Latency):** Dashboard end-to-end data propagation latency must not exceed 1.5 seconds from the database log.
- **NFR-PER-02 (Concurrency):** Must support up to 50 concurrent active connections without degrading performance.
- **NFR-PER-03 (Media):** Database indexing for video files must complete within 2 seconds of reaching cloud storage.

5.2 Security Requirements

- **NFR-SEC-01 (Access Control):** Strict isolation; clients access only their assets, pods access only their active queues.
- **NFR-SEC-02 (Data Lifespan):** Video assets automatically and permanently deleted 7 days after the initial upload.
- **NFR-SEC-03 (Encryption):** All data transfers encrypted in transit utilizing TLS 1.3.

5.3 Usability Requirements

- **NFR-USA-01 (Learnability):** Editors must be able to manage task statuses with less than 10 minutes of onboarding.
- **NFR-USA-02 (Error Prevention):** The portal must display constraints prior to upload and instantly halt violations with clear messaging.

5.4 Scalability & Storage Constraints

- **NFR-SCA-01 (Size Limit):** Hard constraint rejecting any video upload exceeding 50MB to strictly manage AWS prototype tier budgets.
- **NFR-SCA-02 (Extensibility):** Backend logic must be decoupled to allow adding pods (beyond the initial 8) with zero database schema changes.

6. Alternative Solutions & Feasibility Study

Alternative Solutions Analyzed

1. **Trello / Asana:** While visually appealing, these tools lack an automated "brain." They still require the PO to act as a bottleneck, manually dragging and dropping cards based on guesswork.
2. **Enterprise Software (Jira):** Features automated workflows but is overly complex. External clients simply want to upload a video; they will experience friction and confusion if forced to navigate heavy developer-focused UI

Feasibility & Recommendation

- **Technical:** Highly feasible. The required stack (React.js, Node.js, Python, MongoDB) aligns directly with the development team's proficiencies.
- **Economic:** Highly feasible. Utilizing student cloud tiers reduces infrastructure costs to zero for the prototype phase.
- **Operational:** FlowRoute directly addresses the pain points uncovered during Unwir Agency fact-finding, providing an immediate operational upgrade.
- **Recommendation:** FlowRoute is justified as the superior solution, providing the automation lacking in standard Kanban tools without the client-facing bloat of enterprise systems.